

# OPERATING SYSTEMS (UE24CS242B)

## Unit 2 – IPC, Threads and Concurrency

### Lecture 2 - System Calls for Inter-Process Communication

#### Introduction to IPC System Calls

Inter-Process Communication (IPC) system calls allow multiple processes to communicate and synchronize with each other. These mechanisms are essential in multiprogramming environments where processes cooperate to share data, coordinate execution, and improve system efficiency. UNIX-based operating systems provide several IPC mechanisms such as pipes, FIFOs, message queues, semaphores, and shared memory, each suited for different communication needs.

#### Pipes

Pipes are the oldest and simplest form of IPC in UNIX systems and are supported by all UNIX variants. A pipe allows data to flow between two processes in a unidirectional manner, meaning data flows only in one direction (half-duplex). Although some modern systems support full-duplex pipes, programs should not rely on this for portability.

A key limitation of pipes is that they can be used only between processes that share a common ancestor. Typically, a pipe is created by a parent process, which then calls `fork()`, allowing communication between the parent and child processes.

#### `pipe()` System Call

A pipe is created using the `pipe()` system call. When this call is successful, it returns two file descriptors through an array argument:

- `fd[0]` → read end of the pipe
- `fd[1]` → write end of the pipe

Data written to `fd[1]` can be read from `fd[0]`. Internally, the data flows through the kernel buffer, ensuring safe data transfer between processes.

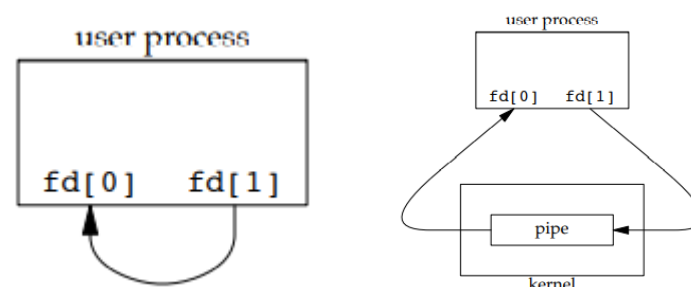


Diagram : Half-duplex pipe showing read end and write end through kernel

#### Pipe Usage Between Parent and Child

In practical usage, after calling `pipe()`, the process calls `fork()` to create a child. Depending on the direction of communication:

- **Parent to Child**
  - Parent closes `fd[0]` (read end)

- Child closes fd[1] (write end)
- **Child to Parent**
  - Parent closes fd[1]
  - Child closes fd[0]

Closing unused ends is important to:

- Prevent deadlocks
- Allow proper detection of end-of-file (EOF)

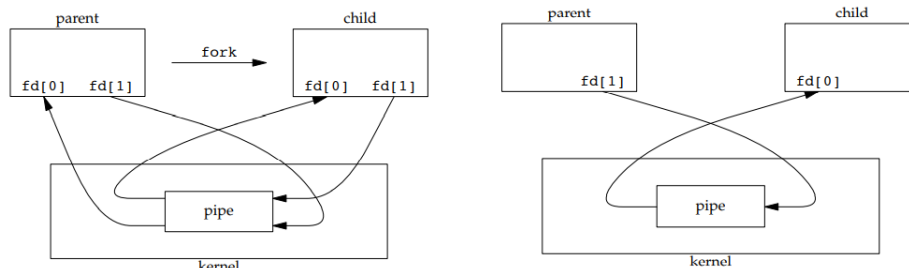


Diagram: Pipe from parent to child process

### **popen() and pclose()**

The popen() and pclose() functions provide a higher-level interface to pipes using the standard I/O library. These functions simplify the process of creating pipes, forking processes, and executing commands.

The popen() function:

- Creates a pipe
- Forks a child process
- Executes a shell command using exec()
- Returns a **file pointer** instead of a file descriptor

Depending on the mode:

- "r" → parent reads the output of the command
- "w" → parent writes input to the command

The pclose() function:

- Closes the stream
- Waits for the command to terminate
- Returns the termination status

This mechanism is commonly used to capture command output or supply input programmatically.

### **FIFOs (Named Pipes)**

FIFOs, also known as named pipes, overcome the major limitation of unnamed pipes by allowing communication between unrelated processes. Unlike pipes, FIFOs exist as special files in the file system.

A FIFO is created using:

- `mkfifo()` or
- `mkfifoat()`

Once created, a FIFO behaves like a regular file and can be accessed using standard file operations such as `open()`, `read()`, `write()`, `close()`, and `unlink()`.

### Uses of FIFOs

FIFOs are mainly used in two scenarios:

1. **Shell pipelines**
  - Allow data transfer between shell commands without using temporary files
2. **Client–Server Communication**
  - Act as rendezvous points where clients and servers exchange data

### Message Queues

A message queue is an IPC mechanism where messages are stored as a linked list inside the kernel. Each message queue is identified by a message queue identifier.

Message queues are created or accessed using `msgget()`. Messages are sent using `msgsnd()` and received using `msgrcv()`.

Each message consists of:

- A type field (positive long integer)
- Message length
- Actual data

Unlike pipes, messages do not have to be received in FIFO order; instead, they can be retrieved based on their type, providing greater flexibility.

### Semaphores

A semaphore is a synchronization tool used to control access to shared resources among multiple processes. Conceptually, a semaphore is a counter that represents the number of available instances of a resource.

When a process wants to access a shared resource:

- It checks the semaphore value
- If the value is positive, it decrements the semaphore and proceeds
- If the value is zero, the process is blocked until the semaphore becomes available

When the process finishes using the resource:

- It increments the semaphore

- Waiting processes are awakened

### **Semaphore System Calls**

To use semaphores, a process must first obtain a semaphore ID using `semget()`. Semaphore operations are performed using:

- `semop()` → atomically performs operations on semaphore sets
- `semctl()` → used for control operations such as initialization, deletion, or querying status

Semaphores are crucial for avoiding **race conditions and ensuring mutual exclusion**.